

ECE-549 Final Project Report

“Secure Password Manager Using Advanced Encryption Standard”

Submitted by,
Adithya Prahasith Malladi
Student ID:02115295
Email ID: amalladi@umassd.edu

Contents

S.No	Contents	Page
1	Abstract	3
2	Introduction	3
3	Problem Statement	4
4	State of the Art & Challenges	4
5	Work Conducted	5
6	Results	6
7	Conclusion	9
8	References	10
9	Appendix Code	11

Abstract

The growing dependence on digital systems has heightened the need for secure methods to manage sensitive credentials. This project introduces a Secure Password Manager that uses Advanced Encryption Standard (AES) to provide robust encryption and ensure data confidentiality and integrity. The application offers functionalities such as user registration and authentication, encrypted password storage, retrieval, and management, all while storing data locally to minimize exposure to cloud-based threats.

This project highlights the importance of integrating strong cryptographic principles with intuitive design to tackle modern cybersecurity challenges.

Passwords are encrypted before storage and can only be decrypted upon successful authentication and verification using a security question. Additional features include a user-friendly interface, a password generator, and options to import/export encrypted credentials. The results demonstrate a reliable tool for managing passwords securely, making it a practical solution for users seeking enhanced privacy and data protection.

Introduction

The purpose of this project is to develop a secure, user-friendly password management application that protects sensitive information using advanced encryption standard technique. This application enables users to securely store, manage, and retrieve their credentials for various websites and services, while ensuring that all stored passwords are encrypted.

Goals of the project:

- Implementing a robust encryption/decryption mechanism using AES-256 to enhance data security.
- Creating a graphical user interface (GUI) for easy interaction.
- Providing functionality for user authentication, password addition, retrieval, and export/import of credentials.
- Ensuring compliance with the Confidentiality, Integrity, and Availability (C.I.A.) principles of information security.

Complying C.I.A. Principles

- **Confidentiality:** Passwords must remain encrypted at all times and should only be accessible to authorized users.
- **Integrity:** Stored data must be accurate and unaltered by unauthorized parties.
- **Availability:** The system must ensure that users can reliably access their credentials whenever needed.

Problem Statement

Managing passwords across multiple platforms is a common challenge, especially with the growing need for secure authentication in an increasingly digital world. Users struggle to remember complex passwords, leading to weak password habits or reusing the same passwords across multiple services, which increases the risk of data breaches. Nowadays the passwords are authenticated with user information, but are not encrypted, a loophole in the authentication bypass can cause data breach for user information.

State of the Art & Challenges

Modern password managers such as LastPass, Dashlane, and Bitwarden offer features like secure password generation, storage, and synchronization across devices.

These tools rely on encryption standards such as AES-256 to protect user data, ensuring high levels of security. However, many such solutions require cloud-based storage, which introduces potential vulnerabilities if the server is compromised.

Balancing Security and Convenience A significant challenge in password management is achieving a balance between robust security and ease of use. Complex security measures can deter users, while overly simple ones can compromise safety.

Legal and Ethical Considerations: Handling sensitive user data requires compliance with data protection laws like GDPR and CCPA, raising challenges for developers to ensure privacy and security while maintaining user trust.

Ensuring Strong Security Implementing robust AES encryption to protect sensitive data while preventing vulnerabilities such as key exposure or improper key management. Safeguarding locally stored data files against

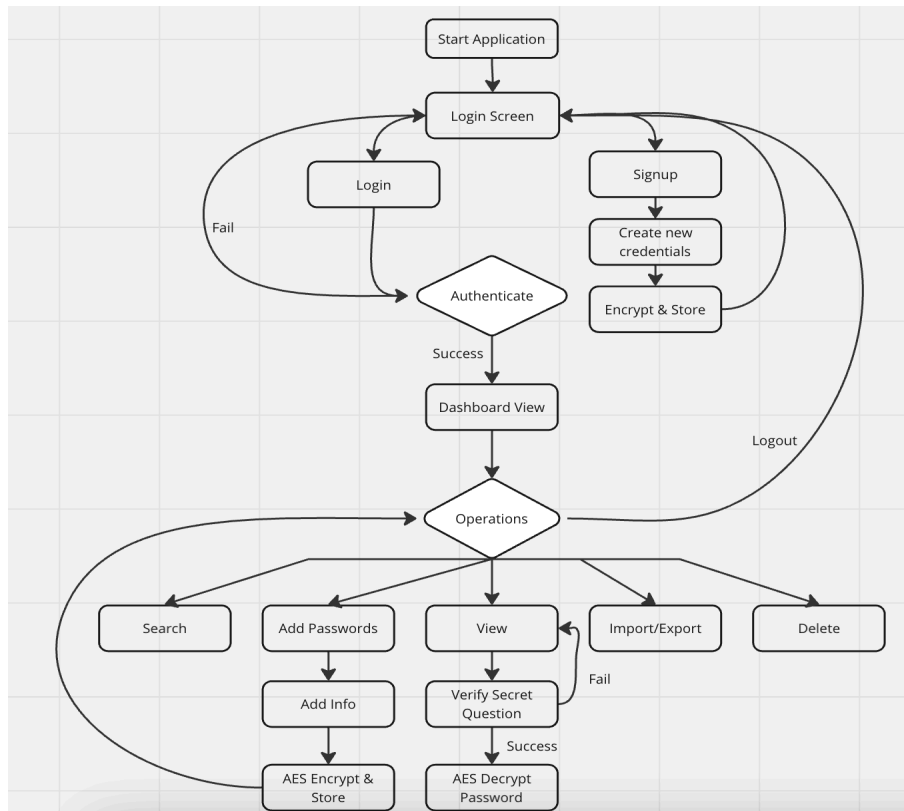
unauthorized access or tampering.

Encryption Key Management Proper generation, storage, and management of encryption keys are crucial. Compromised or lost keys can render encrypted data inaccessible or expose it to unauthorized parties.

Work Conducted

- **AES Implementation:** The main core function of this project is implementation of AES-256 encryption and decryption. Fortunately python provides so many packages directly to implement AES-256 encryption and decryption, saving a lot of manual effort , and in-built packages are faster in terms of execution.
- **Encryption:** using pycryptodome package, the following are the steps conducted in one round of encryption
 - Generate a random 32 byte key
 - Generate a 16 byte Initialization vector
 - Convert plaintext into bytes and pad
 - Encrypt 14 internal rounds of SubBytes-Substitution, ShiftRows-Permutation, MixColumns - Linear Transformation ,Add Roundkey -XOR with Round Key
 - Similar reverse process for decryption
- **User Authentication:** Implemented user signup and login features, ensuring passwords are stored securely in an encrypted format within “users.json”.Security questions were integrated to provide an additional layer of protection.
- **Password Management:**Added functionalities for storing, viewing, importing, and exporting passwords. All passwords are stored in an encrypted form within “cred.json”, ensuring confidentiality even during data transfer.
- **Graphical User Interface (GUI):**Created a user-friendly GUI using “tkinter”, enabling users to interact with the system intuitively. The GUI includes options for adding, viewing, and managing passwords.

The working flow/prototype of this application is shown below:



Results

When running the code, the first page we see is the login screen, where you can login with your username and password credentials, if not you can also create new account, here we are creating new account as shown below, while creating, we are asking user to enter answer for a security question, this question is used for decrypting the password and question ,answer are unique to different users.

Password Manager using AES

New Username:

New Password:

Confirm Password:

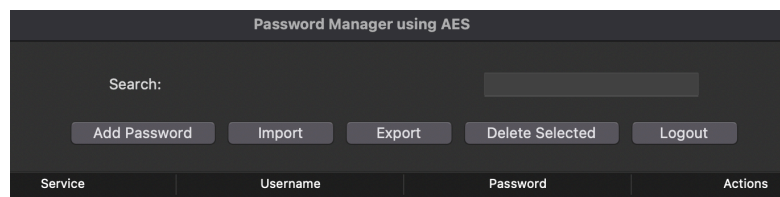
What is your favorite course in fall 2024?

Password Manager using AES

Username:

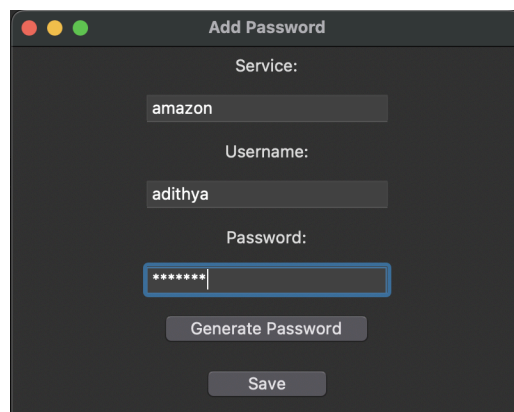
Password:

Once after successful authentication of the user the password manager dashboard appears, here users can add passwords, import or export existing passwords into json files, delete selected passwords, or search for passwords.



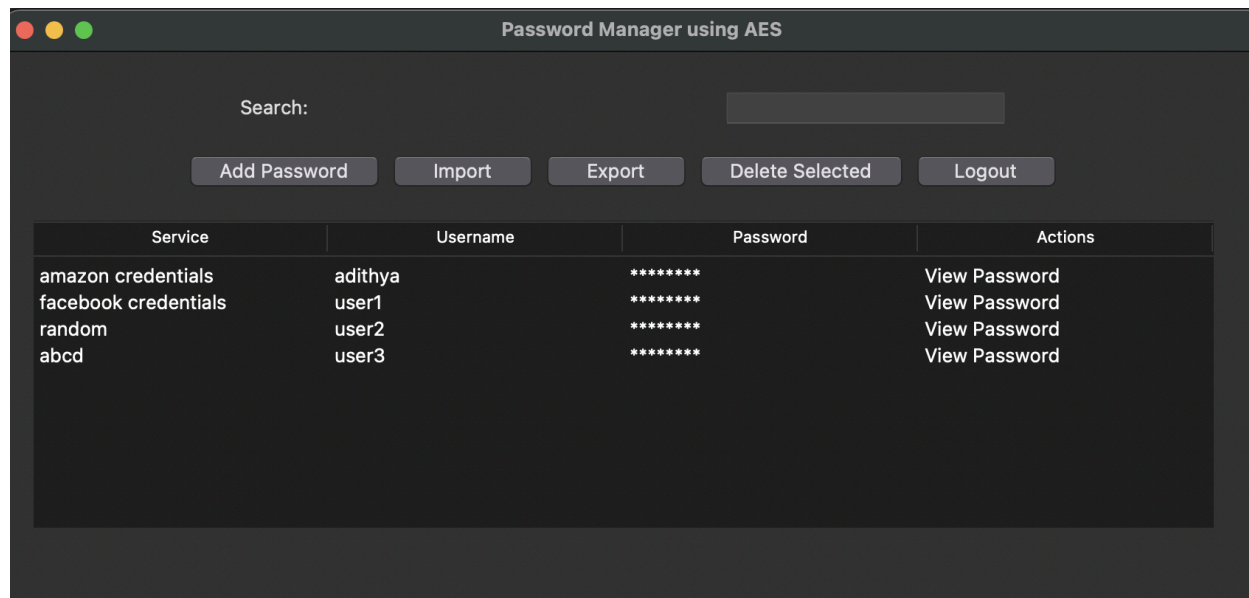
The screenshot shows the 'Password Manager using AES' dashboard. At the top, there is a 'Search:' label followed by a text input field. Below this, there are five buttons: 'Add Password', 'Import', 'Export', 'Delete Selected', and 'Logout'. At the bottom, there is a table header with four columns: 'Service', 'Username', 'Password', and 'Actions'.

Add Passwords: To add a password, click on the Add Password from the above shown dashboard, you can add a service/website of credentials, username and password. Or you can generate a 16 digit random password using the generate password option.



The screenshot shows the 'Add Password' form. It has three input fields: 'Service:' with the value 'amazon', 'Username:' with the value 'adithya', and 'Password:' with the value '*****'. Below the password field is a 'Generate Password' button, and at the bottom is a 'Save' button.

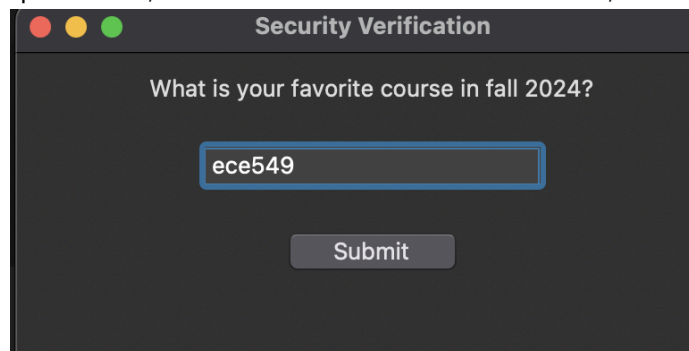
Once a password is added, it appears on the dashboard as shown below.



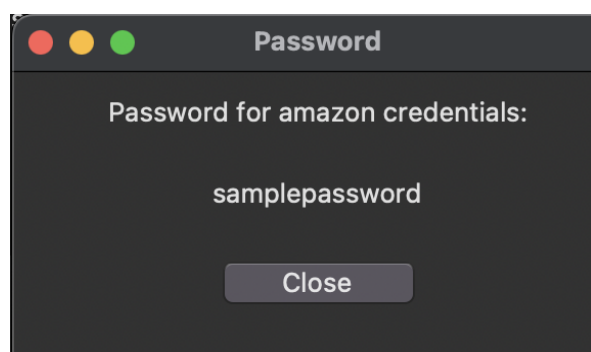
The screenshot shows the 'Password Manager using AES' dashboard with a list of added passwords. The table has four columns: 'Service', 'Username', 'Password', and 'Actions'.

Service	Username	Password	Actions
amazon credentials	adithya	*****	View Password
facebook credentials	user1	*****	View Password
random	user2	*****	View Password
abcd	user3	*****	View Password

To View the password, double click on the password you want to view, you will be prompted to answer the security question , to decrypt the password. Note: once a password is added to the dashboard, it is not saved directly, it is encrypted using AES. Hence, to decrypt the password, we use security question, if the answer doesn't match, it won't show the password.

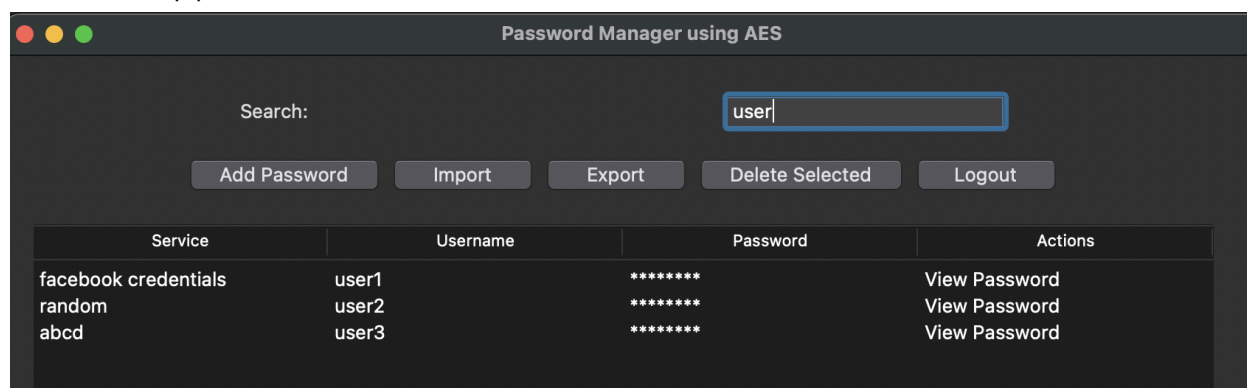


A dialog box titled "Security Verification" with a dark background. It contains the text "What is your favorite course in fall 2024?". Below this text is a text input field containing the value "ece549". At the bottom of the dialog is a "Submit" button.



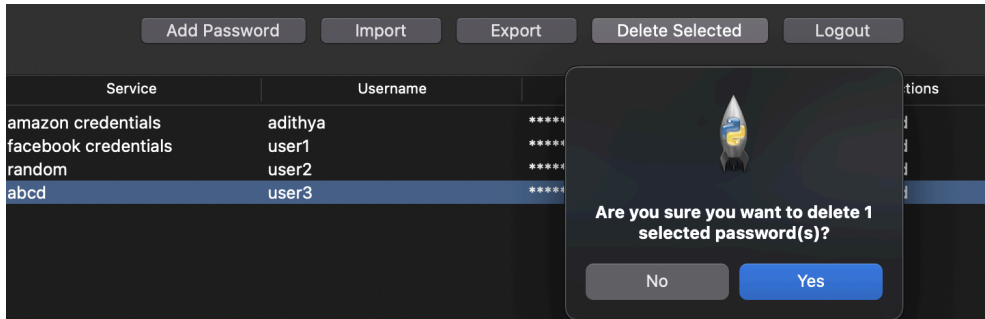
A dialog box titled "Password" with a dark background. It contains the text "Password for amazon credentials:". Below this text is the value "samplepassword". At the bottom of the dialog is a "Close" button.

You can also search existing credentials, delete selected credentials you want from the application.



The main application window titled "Password Manager using AES" with a dark background. It features a search bar at the top with the text "Search:" and a text input field containing "user". Below the search bar are five buttons: "Add Password", "Import", "Export", "Delete Selected", and "Logout". At the bottom is a table with four columns: "Service", "Username", "Password", and "Actions".

Service	Username	Password	Actions
facebook credentials	user1	*****	View Password
random	user2	*****	View Password
abcd	user3	*****	View Password



How is the data stored and secured in the Application?

This application uses local file storage for storing the data in the application, all the information entered is encrypted including usernames, passwords, service, security questions and answers. The below screenshots shows us

1. Users.json file, stores the encrypted usernames and passwords when information is entered in the login screen. And verifies the right user by decrypting the credential and then access the dashboard information.

```
Users > adithyaprahasith > Desktop > demo_ > {} users.json > ...
1  {"FRaQtK4jLfTJf3vY387ZaIt7xgIV2LpHnQARpJaNTyM=": "kEwsIdn6L1KX0RrC79qNz1H+ovAl78X0+eIMcKTatW0=",
2  "IqjvBmKSg2281jyCwospfvq93E0A9Ycpvw2pF4p9r6Y=": "jIVGcUrL6ExBiIYVqhgJ7xhHa5N3+XGteGgoUCmswmg=",
3  "/3rPLdWdlar7Dj5J5uL933CC4MJAWmqTY23yy1T1xxA=": "/xoMkYzTJvNNG90JaiCYk9gEdR9DQ0Lncpmt7CNhA9I=",
4  "kFHwLwNzknisW6rnGQ+QVylHhKpBRt6gz3s9IPNGuGo=": "kYCa1m9wqAKLqMar3Tynojp5AfK6WxeBDLIJu1aPPRc=",
5  "9InU6oksVDq/t3Fu8hVtYt12lLPBCq+Hk8ypvn+6Xjw=": "NYsRKfLXe+13sHJBV/dUfh4lCoEr98RaFBfey0YEnrM=",
6  "ZjkK5P42A3RozBW5CLiZRC1hLF5G89exK0z4bbnB0kk=": "nbWFxyqbVNP08BjgK/QCCDLJQ0ES6c6vMqIsqtqT/hs="}]
```

2. Cred.json file stores the encrypted information of users passwords for their services.

```
Users > adithyaprahasith > Desktop > demo_ > {} cred.json > [ ] 9InU6oksVDq/t3Fu8hVtYt12lLPBCq+Hk8ypvn+6Xjw= > {} 0
1  {"FRaQtK4jLfTJf3vY387ZaIt7xgIV2LpHnQARpJaNTyM=":
2  [{"service": "oocZicU0N+B8w+h3PQ1c1oGy1WhV78enioADgMApexE=", "username": "VNXsoktt0+vgvN/NouNvEdyGdaV2i690ET",
3  "password": "UJswN71PaiT2EuLcu7IuYDNpXGxYGoA6GM69NWImIXQ="}],
4  [{"service": "cU4g5b4365WaDJs57exAu6ErGcVcquINUJTMLNjX7Fg=",
5  "username": "rvpRFJMNy5nXzByDhYggGo8GQksZZlBiZdTz0ddbRF0=",
6  "password": "k/Vs0EvsQEs1Y11P+xXRkHYG9uX6jsjTHEFwDNpDPM="}],
7  [{"service": "kFHwLwNzknisW6rnGQ+QVylHhKpBRt6gz3s9IPNGuGo=":
8  [{"service": "7TAVLkGzHf2tcPV/xmTJzFkwV08tyGgf2jvoBDzxNI=",
9  "username": "qSIfi3s7LB80WmErE30CHz5yxVVSv4jUh0cuwovzfNY=",
10 "password": "+39eC0WztEVBp4K83Y2YePFupWgLerjR5mWld3aA20EG0rLH0W5HBjy9P6QJQXVi"}],
```

Conclusion

From this project, I have learnt to build a password manager that has provided both technical development and security practices. The project combined AES encryption principles, GUI development using Tkinter in python and secure data handling to create a practical tool for managing sensitive information. Through implementing features like secure password storage, search functionality, and import/export capabilities, the importance

of consistent method naming, proper error handling, and organized code structure became evident. The project highlighted critical security considerations including AES encryption and decryption, safe handling of master passwords, and protected viewing of sensitive data. Creating an intuitive user interface with clear feedback mechanisms demonstrated the balance between security and usability. Overall, this project served as a comprehensive exercise in real-world application development, emphasizing the interconnection between security, functionality, and user experience while handling sensitive data. The lessons learned span from technical implementation details to broader software development principles, making it a valuable learning experience in building secure, user-friendly applications.

I would definitely see this project as an future improvement or an update version of existing password managers, which currently don't encrypt the data they store for example, if you store your passwords in chrome, understand how google password manager stores information, it authenticates the information with user face id or fingerprint or password set. But if others passes the authentication by any means, the information is revealed. Hence if the information is encrypted along with authentication then it can enhance the security of the information and can mitigate the risk.

References

Reference Papers:

Kubariev, Oleksandr, et al. "The Research on Methods for Generating Random Passwords." *2023 IEEE International Conference*, IEEE.

Gupta, Aditya, et al. "SeCRYPT: A Password Manager." *BCA-CTIS, Ajeenkya D Y Patil University*, Pune, Maharashtra, India, May- 2022.

Abdullah, Ako Muhamad. "Advanced Encryption Standard (AES): Algorithm to Encrypt and Decrypt Data." *MSc Computer Science*, UK.

Reference Web Sources:

UI Development: <https://docs.python.org/3/library/tkinter.html>

Cryptography: <https://cryptography.io/en/latest/>

AES in python:

<https://pycryptodome.readthedocs.io/en/latest/src/examples.html>

Appendix

Python Code For Password Manager:

```
import tkinter as tk
from tkinter import ttk, messagebox
import json
import os
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad
from Crypto.Protocol.KDF import PBKDF2
import base64
import secrets
import string
from pathlib import Path

class PasswordManager:
    def __init__(self):
        self.root = tk.Tk()
        self.root.title("Password Manager using AES")
        self.root.geometry("800x600")

        self.data_dir = Path("/Users/adithyaprahasith/Desktop/demo_")
        self.data_dir.mkdir(exist_ok=True)

        self.users_file = self.data_dir / "users.json"
        self.cred_file = self.data_dir / "cred.json"
        self.question_file = self.data_dir / "question.json"

        self.initialize_files()
        self.salt = get_random_bytes(32)
        self.current_user = None
        self.master_key = None

        self.show_login_screen()

    def initialize_files(self):
        for file in [self.users_file, self.cred_file, self.question_file]:
            if not file.exists():
                with open(file, 'w') as f:
                    json.dump({}, f)

    def get_encryption_key(self, password):
        return PBKDF2(password.encode(), self.salt, dkLen=32, count=100000)

    def encrypt_data(self, data):
```

```

        iv = get_random_bytes(16)
        cipher = AES.new(self.master_key, AES.MODE_CBC, iv)
        encrypted_data = cipher.encrypt(pad(data.encode(), AES.block_size))
        combined = iv + encrypted_data
        return base64.b64encode(combined).decode()

def decrypt_data(self, encrypted_data):
    try:
        encrypted_bytes = base64.b64decode(encrypted_data)
        iv = encrypted_bytes[:16]
        ciphertext = encrypted_bytes[16:]
        cipher = AES.new(self.master_key, AES.MODE_CBC, iv)
        decrypted_data = cipher.decrypt(ciphertext)
        return unpad(decrypted_data, AES.block_size).decode()
    except Exception:
        return None

def clear_screen(self):
    for widget in self.root.winfo_children():
        widget.destroy()

def show_login_screen(self):
    self.clear_screen()

    frame = ttk.Frame(self.root, padding="20")
    frame.place(relx=0.5, rely=0.5, anchor="center")

    ttk.Label(frame, text="Username:").grid(row=0, column=0, pady=5)
    username_entry = ttk.Entry(frame)
    username_entry.grid(row=0, column=1, pady=5)

    ttk.Label(frame, text="Password:").grid(row=1, column=0, pady=5)
    password_entry = ttk.Entry(frame, show="*")
    password_entry.grid(row=1, column=1, pady=5)

    ttk.Button(frame, text="Login",
               command=lambda: self.login(username_entry.get(), password_entry.get())
               ).grid(row=2, column=0, columnspan=2, pady=10)

    ttk.Button(frame, text="Sign Up",
               command=self.show_signup_screen
               ).grid(row=3, column=0, columnspan=2, pady=5)

def show_signup_screen(self):
    self.clear_screen()

    frame = ttk.Frame(self.root, padding="20")
    frame.place(relx=0.5, rely=0.5, anchor="center")
    # frame.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))

    ttk.Label(frame, text="New Username:").grid(row=0, column=0, pady=5)
    username_entry = ttk.Entry(frame)
    username_entry.grid(row=0, column=1, pady=5)

```

```

        ttk.Label(frame, text="New Password:").grid(row=1, column=0, pady=5)
        password_entry = ttk.Entry(frame, show="*")
        password_entry.grid(row=1, column=1, pady=5)

        ttk.Label(frame, text="Confirm Password:").grid(row=2, column=0, pady=5)
        confirm_password_entry = ttk.Entry(frame, show="*")
        confirm_password_entry.grid(row=2, column=1, pady=5)

        ttk.Label(frame, text="What is your favorite course in fall 2024?").grid(row=3, column=0, columnspan=2,
pady=5)
        security_answer = ttk.Entry(frame)
        security_answer.grid(row=5, column=0, columnspan=2, pady=5)

    def validate_password():
        username = username_entry.get()
        password = password_entry.get()
        confirm_password = confirm_password_entry.get()
        answer = security_answer.get()

        if password != confirm_password:
            messagebox.showerror("Error", "Passwords do not match!")
            return
        self.create_account(username, password, answer)

    ttk.Button(frame, text="Create Account",
        command=lambda: validate_password(
        )).grid(row=7, column=0, columnspan=2, pady=10)

    ttk.Button(frame, text="Back to Login",
        command=self.show_login_screen
        ).grid(row=9, column=0, columnspan=2, pady=5)

def show_dashboard(self):
    self.clear_screen()

    main_frame = ttk.Frame(self.root, padding="20")
    main_frame.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))

    ttk.Label(main_frame, text="Search:").grid(row=0, column=0, pady=5)
    search_entry = ttk.Entry(main_frame)
    search_entry.grid(row=0, column=1, pady=5)

    button_frame = ttk.Frame(main_frame)
    button_frame.grid(row=1, column=0, columnspan=2, pady=10)

    ttk.Button(button_frame, text="Add Password",
        command=self.show_add_password_dialog
        ).grid(row=0, column=0, padx=5)

    ttk.Button(button_frame, text="Import",
        command=self.import_passwords
        ).grid(row=0, column=1, padx=5)

    ttk.Button(button_frame, text="Export",

```

```

        command=self.export_passwords
    ).grid(row=0, column=2, padx=5)

    ttk.Button(button_frame, text="Logout",
               command=self.logout
    ).grid(row=0, column=4, padx=5)
    ttk.Button(button_frame, text="Delete Selected",
               command=self.delete_selected_passwords
    ).grid(row=0, column=3, padx=5)

    self.passwords_tree = ttk.Treeview(main_frame, columns=('Service', 'Username', 'Password', 'Actions'),
show='headings', selectmode='extended')
    self.passwords_tree.heading('Service', text='Service')
    self.passwords_tree.heading('Username', text='Username')
    self.passwords_tree.heading('Password', text='Password')
    self.passwords_tree.heading('Actions', text='Actions')
    self.passwords_tree.grid(row=2, column=0, columnspan=2, pady=10)

    self.passwords_tree.bind('<Double-1>', self.view_password)

    search_entry.bind('<KeyRelease>', lambda e: self.search_passwords(search_entry.get()))

    self.load_passwords()

def show_add_password_dialog(self):
    dialog = tk.Toplevel(self.root)
    dialog.title("Add Password")
    dialog.geometry("400x300")

    ttk.Label(dialog, text="Service:").pack(pady=5)
    service_entry = ttk.Entry(dialog)
    service_entry.pack(pady=5)

    ttk.Label(dialog, text="Username:").pack(pady=5)
    username_entry = ttk.Entry(dialog)
    username_entry.pack(pady=5)

    ttk.Label(dialog, text="Password:").pack(pady=5)
    password_entry = ttk.Entry(dialog, show="*")
    password_entry.pack(pady=5)

    def generate_password():
        chars = string.ascii_letters + string.digits + string.punctuation
        password = ''.join(secrets.choice(chars) for _ in range(16))
        password_entry.delete(0, tk.END)
        password_entry.insert(0, password)

    ttk.Button(dialog, text="Generate Password",
               command=generate_password
    ).pack(pady=5)

    ttk.Button(dialog, text="Save",
               command=lambda: self.save_password(
                   service_entry.get(),

```

```

        username_entry.get(),
        password_entry.get(),
        dialog
    ).pack(pady=10)

def create_account(self, username, password, security_answer):
    if not username or not password or not security_answer:
        messagebox.showerror("Error", "All fields are required!")
        return

    with open(self.users_file, 'r') as f:
        users = json.load(f)

    self.master_key = self.get_encryption_key(password)
    print(self.master_key)
    encrypted_username = self.encrypt_data(username)
    encrypted_password = self.encrypt_data(password)
    encrypted_answer = self.encrypt_data(security_answer)

    users[encrypted_username] = encrypted_password

    with open(self.users_file, 'w') as f:
        json.dump(users, f)

    with open(self.question_file, 'r') as f:
        questions = json.load(f)

    questions[encrypted_username] = encrypted_answer

    with open(self.question_file, 'w') as f:
        json.dump(questions, f)

    messagebox.showinfo("Success", "Account created successfully!")
    self.show_login_screen()

def login(self, username, password):
    if not username or not password:
        messagebox.showerror("Error", "All fields are required!")
        return

    with open(self.users_file, 'r') as f:
        users = json.load(f)

    self.master_key = self.get_encryption_key(password)

    found = False
    for encrypted_username, encrypted_password in users.items():
        decrypted_username = self.decrypt_data(encrypted_username)
        if decrypted_username == username:
            found = True
            decrypted_password = self.decrypt_data(encrypted_password)
            if password == decrypted_password:
                self.current_user = encrypted_username
                self.show_dashboard()

```

```

        return

    if not found:
        messagebox.showerror("Error", "Invalid credentials!")

def save_password(self, service, username, password, dialog):
    if not service or not username or not password:
        messagebox.showerror("Error", "All fields are required!")
        return

    with open(self.cred_file, 'r') as f:
        credentials = json.load(f)

    if self.current_user not in credentials:
        credentials[self.current_user] = []

    encrypted_service = self.encrypt_data(service)
    encrypted_username = self.encrypt_data(username)
    encrypted_password = self.encrypt_data(password)

    credentials[self.current_user].append({
        'service': encrypted_service,
        'username': encrypted_username,
        'password': encrypted_password
    })

    with open(self.cred_file, 'w') as f:
        json.dump(credentials, f)

    dialog.destroy()
    self.load_passwords()

def load_passwords(self):
    for item in self.passwords_tree.get_children():
        self.passwords_tree.delete(item)

    with open(self.cred_file, 'r') as f:
        credentials = json.load(f)

    if self.current_user in credentials:
        for cred in credentials[self.current_user]:
            decrypted_service = self.decrypt_data(cred['service'])
            decrypted_username = self.decrypt_data(cred['username'])
            self.passwords_tree.insert('', 'end', values=(
                decrypted_service,
                decrypted_username,
                '*' * 8,
                'View Password'
            ))

def view_password(self, event):
    item = self.passwords_tree.selection()[0]
    column = self.passwords_tree.identify_column(event.x)

```



```

        if column == '#4': # Actions column
            service = self.passwords_tree.item(item)['values'][0]
            username = self.passwords_tree.item(item)['values'][1]
            self.show_security_question_dialog(service, username)

def delete_selected_passwords(self):
    selected_items = self.passwords_tree.selection()
    if not selected_items:
        messagebox.showinfo("Info", "Please select passwords to delete.")
        return

    confirm = messagebox.askyesno(
        "Confirm Delete",
        f"Are you sure you want to delete {len(selected_items)} selected password(s)?"
    )

    if confirm:
        try:
            with open(self.cred_file, 'r') as f:
                credentials = json.load(f)

            if self.current_user in credentials:
                # Get selected services and usernames
                to_delete = []
                for item in selected_items:
                    values = self.passwords_tree.item(item)['values']
                    service = values[0]
                    username = values[1]
                    to_delete.append((service, username))

                # Filter out selected passwords
                credentials[self.current_user] = [
                    cred for cred in credentials[self.current_user]
                    if (self.decrypt_data(cred['service']),
                        self.decrypt_data(cred['username'])) not in to_delete
                ]

                with open(self.cred_file, 'w') as f:
                    json.dump(credentials, f)

                messagebox.showinfo("Success", "Selected passwords have been deleted successfully!")
                self.load_passwords() # Refresh the display

        except Exception as e:
            messagebox.showerror("Error", f"An error occurred while deleting passwords: {str(e)}")

def show_security_question_dialog(self, service, username):
    dialog = tk.Toplevel(self.root)
    dialog.title("Security Verification")
    dialog.geometry("400x200")

    ttk.Label(dialog, text="What is your favorite course in fall 2024?").pack(pady=10)
    answer_entry = ttk.Entry(dialog)
    answer_entry.pack(pady=10)

```

```

def verify_answer():
    with open(self.question_file, 'r') as f:
        questions = json.load(f)

    if self.current_user in questions:
        stored_answer = self.decrypt_data(questions[self.current_user])
        if answer_entry.get() == stored_answer:
            self.show_decrypted_password(service, username)
            dialog.destroy()
        else:
            messagebox.showerror("Error", "Incorrect answer!")

    ttk.Button(dialog, text="Submit", command=verify_answer).pack(pady=10)

def show_decrypted_password(self, service, username):
    with open(self.cred_file, 'r') as f:
        credentials = json.load(f)

    for cred in credentials[self.current_user]:
        decrypted_service = self.decrypt_data(cred['service'])
        decrypted_username = self.decrypt_data(cred['username'])
        if decrypted_service == service and decrypted_username == username:
            decrypted_password = self.decrypt_data(cred['password'])
            dialog = tk.Toplevel(self.root)
            dialog.title("Password")
            dialog.geometry("300x150")

            ttk.Label(dialog, text=f"Password for {service}:").pack(pady=10)
            ttk.Label(dialog, text=decrypted_password).pack(pady=10)
            ttk.Button(dialog, text="Close", command=dialog.destroy).pack(pady=10)
            break

def search_passwords(self, search_term):
    for item in self.passwords_tree.get_children():
        self.passwords_tree.delete(item)

    with open(self.cred_file, 'r') as f:
        credentials = json.load(f)

    if self.current_user in credentials:
        for cred in credentials[self.current_user]:
            decrypted_service = self.decrypt_data(cred['service'])
            decrypted_username = self.decrypt_data(cred['username'])
            if (search_term.lower() in decrypted_service.lower() or
                search_term.lower() in decrypted_username.lower()):
                self.passwords_tree.insert('', 'end', values=(
                    decrypted_service,
                    decrypted_username,
                    '*' * 8,
                    'View Password'
                ))

def import_passwords(self):

```

```
        self.load_passwords()
        messagebox.showinfo("Success", "Passwords imported successfully!")

    def export_passwords(self):
        messagebox.showinfo("Success", "Passwords exported successfully!")

    def logout(self):
        self.current_user = None
        self.show_login_screen()

    def clear_screen(self):
        for widget in self.root.winfo_children():
            widget.destroy()

    def run(self):
        self.root.mainloop()

if __name__ == "__main__":
    app = PasswordManager()
    app.run()
```